

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. Addition (`matrix_a + matrix_b`)
2. Subtraction (`matrix_a - matrix_b`)
3. Element-wise Multiplication (`matrix_a * matrix_b`)
4. Matrix Multiplication (`matrix_a · matrix_b`)
5. Transpose of Matrix A

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Sample Test Cases

+

Explorer

matrixOper...

▶

Submit

Debugger

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int,
6 input().split())) for i in range(3)])
7
8 print("Enter Matrix B:")
9 matrix_b = np.array([list(map(int,
10 input().split())) for i in range(3)])
11
12 # Addition
13 print("Addition (A + B):")
14 a = matrix_a + matrix_b
15 print(a)
16 # Subtraction
17 print("Subtraction (A - B):")
18 b = matrix_a - matrix_b
19 print(b)
20 # Multiplication (element-wise)
21 print("Element-wise Multiplication
22 (A * B):")
23 c = matrix_a * matrix_b
24 print(c)
25 # Matrix multiplication (dot product)
26 print("A dot B:")
27 d = np.dot(matrix_a, matrix_b)
28 print(d)
29 # Transpose
30 print("Transpose of A:")
31 t = np.transpose(matrix_a)
32 print(t)
```

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

+

Explorer

stacking.py

🔍

Submit

Debugger

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int,
6 input().split())) for i in range(3)])
7
8 print("Enter Array2:")
9 arr2 = np.array([list(map(int,
10 input().split())) for i in range(3)])
11
12 # Perform horizontal stacking (hstack)
13 h_stack = np.hstack((arr1, arr2))
14
15 # Perform vertical stacking (vstack)
16 v_stack = np.vstack((arr1, arr2))
17 print("Horizontal Stack:")
18 print(h_stack)
19 print("Vertical Stack:")
20 print(v_stack)
```

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases

+

Explorer

customSe...

▶

Submit

Debugger

```
1 import numpy as np
2
3 # Take user input for the start,
4 # stop, and step of the sequence
5 start = int(input())
6 stop = int(input())
7 step = int(input())
8
9 # Generate the sequence using
10 np.arange()
11 arr = np.arange(start, stop, step)
12 # Print the generated sequence
13 print(arr)
```


You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A_i OR B_i).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

+

```

10  # -> prod_result =
11
12  # -> # Statistical Operations
13  # -> mean_A =
14  # -> median_A =
15  # -> std_dev_A =
16
17  # -> # Bitwise Operations
18  # -> and_result =
19  # -> or_result =
20  # -> xor_result =
21
22  # ... # Output results with one
    space between each element
23  # -> print("Element-wise Sum:", '
    '.join(map(str, sum_result)))
24  # -> print("Element-wise
    Difference:", ' '.join(map(str,
    diff_result)))
25  # -> print("Element-wise Product:", '
    '.join(map(str, prod_result)))
26  # ...
27  # -> print(f"Mean of A: {mean_A}")
28  # -> print(f"Median of A: {median_A}")
29  # -> print(f"Standard Deviation of A:
    {std_dev_A}")
30  # ...
31  # -> print("Bitwise AND:", '
    '.join(map(str, and_result)))
32  # -> print("Bitwise OR:", '
    '.join(map(str, or_result)))
33  # -> print("Bitwise XOR:", '
    '.join(map(str, xor_result)))
34
35  # A = list(map(int,
    input().split())) # Elements of
    array A
36  # B = list(map(int,
    input().split())) # Elements of
    array B
37  # array_operations(A, B)
38  import numpy as np
39
40  def array_operations(A, B):
41      a = np.array(A)
42      b = np.array(B)
43
44      # Arithmetic Operations
45      print("Element-wise Sum:", *a +
        b)
46      print("Element-wise
        Difference:", *a - b)
47      print("Element-wise Product:",
        *a * b)
48
49      # Statistical Operations
50      print("Mean of A:", np.mean(a))
51      print("Median of A:",
        np.median(a))
52      print("Standard Deviation of
        A:", np.std(a))
53
54      # Bitwise Operations
55      print("Bitwise AND:",
        *np.bitwise_and(a, b))
56      print("Bitwise OR:",
        *np.bitwise_or(a, b))
57      print("Bitwise XOR:",
        *np.bitwise_xor(a, b))
58
59      # Input reading
60      A = list(map(int, input().split()))
61      B = list(map(int, input().split()))
62      array_operations(A, B)

```

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
Copy array: <copy_array>
```

Sample Test Cases

+

Explorer

copyAndvi...

🔍

Submit

Debugger

```
1 import numpy as np
2
3 inputlist =
4 list(map(int,input().split(" ")))
5
6 # Original array
7 original_array = np.array(inputlist)
8
9 # Create a view
10 view_array = original_array.view()
11
12 # Create a copy
13 copy_array = original_array.copy()
14
15 # Modify the view
16 view_array[0] = 99
17 print("Original array after
18 modifying view:", original_array)
19 print("View array:", view_array)
20
21 # Modify the copy
22 copy_array[1] = 88
23 print("Original array after
24 modifying copy:", original_array)
25 print("Copy array:", copy_array)
```

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching:** Find the indices where search_value appears in array1 and print these indices.
2. **Counting:** Count how many times count_value appears in array1 and print the count.
3. **Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
4. **Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

Sample Test Cases

+

Explorer

arrayOpera...

▶

Submit

Debugger

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int,
5 input().split()))))
6
7 # Searching
8 search_value = int(input("Value to
9 search: "))
10 count_value = int(input("Value to
11 count: "))
12 broadcast_value = int(input("Value
13 to add: "))
14
15 # Find indices where value matches
16 in array1
17 indices = np.where(array1 ==
18 search_value)[0]
19 print(indices)
20 # Count occurrences in array1
21 count = np.count_nonzero(array1 ==
22 count_value)
23 print(count)
24 # Broadcasting addition
25 broadcasted_array = array1 +
26 broadcast_value
27 print(broadcasted_array)
28 # Sort the first array
29 sorted_array = np.sort(array1)
30 print(sorted_array)
```


Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject:** Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored ≥ 90 :** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

+

```

5 import numpy as np
6
7 a = np.loadtxt("Sample.csv",
8               delimiter=',', skiprows=1)
9
10 # 1. Print all student details
11 print("All student Details:\n", a)
12
13 # 2. Total students
14 print("Total Students:", len(a))
15
16 # 3. All student roll numbers
17 print("All Student Roll Nos", a[:, 0])
18
19 # 4. Subject 1 marks
20 print("Subject 1 Marks", a[:, 1])
21
22 # 5. Min marks in Subject 2
23 print("Min marks in Subject 2",
24       np.min(a[:, 2]))
25
26 # 6. Max marks in Subject 3
27 print("Max marks in Subject 3",
28       np.max(a[:, 3]))
29
30 # 7. All subject marks
31 print("All subject marks:", a[:, 1:])
32
33 # 8. Total marks of each student
34 total_marks = np.sum(a[:, 1:],
35                      axis=1)
36 print("Total Marks", total_marks)
37
38 # 9. Average marks of each student
39 avg_student = np.round(np.mean(a[:,
40                               1:], axis=1), 1)
41 print(avg_student)
42
43 # 10. Average marks of each subject
44 avg_subject = np.round(np.mean(a[:,
45                                1:], axis=0), 1)
46 print("Average Marks of each
47       subject", avg_subject)
48
49 # 11. Average marks of Subject 1 and
50       Subject 2
51 print("Average Marks of S1 and S2",
52       avg_subject[:2])
53
54 # 12. Average marks of Subject 1 and
55       Subject 3
56 print("Average Marks of S1 and S3",
57       avg_subject[:2])
58
59 # 13. Roll no with maximum marks in
60       Subject 3
61 print("Roll no who got maximum marks
62       in Subject 3", a[np.argmax(a[:, 3],
63                                  0)])
64
65 # 14. Roll no with minimum marks in
66       Subject 2
67 print("Roll no who got minimum marks
68       in Subject 2", a[np.argmin(a[:, 2],
69                                  0)])
70
71 # 15. Roll no of students who scored
72       24 in Subject 2
73 print("Roll no who got 24 marks in
74       Subject 2", a[a[:, 2] == 24, :1])
75
76 # 16. Count of students with less
77       than 40 in Subject 1

```

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject:** Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored ≥ 90 :** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

+

```

39 avg_subject = np.round(np.mean(a[:,
40 1:], axis=0), 1)
41 print("Average Marks of each
42 subject", avg_subject)
43
44 # 11. Average marks of Subject 1 and
45 Subject 2
46 print("Average Marks of S1 and S2",
47 avg_subject[:2])
48
49 # 12. Average marks of Subject 1 and
50 Subject 3
51 print("Average Marks of S1 and S3",
52 avg_subject[:,2])
53
54 # 13. Roll no with maximum marks in
55 Subject 3
56 print("Roll no who got maximum marks
57 in Subject 3", a[np.argmax(a[:, 3]),
58 0])
59
60 # 14. Roll no with minimum marks in
61 Subject 2
62 print("Roll no who got minimum marks
63 in Subject 2", a[np.argmin(a[:, 2]),
64 0])
65
66 # 15. Roll no of students who scored
67 24 in Subject 2
68 print("Roll no who got 24 marks in
69 Subject 2", a[a[:, 2] == 24, :1])
70
71 # 16. Count of students with less
72 than 40 in Subject 1
73 print("Count of students who got
74 marks in Subject 1 < 40",
75 np.sum(a[:, 1] < 40))
76
77 # 17. Count of students with more
78 than 90 in Subject 2
79 print("Count of students who got
80 marks in Subject 2 > 90:",
81 np.sum(a[:, 2] > 90))
82
83 # 18. Count of students  $\geq 90$  in
84 each subject
85 print("Count of students in each
86 subject who got marks  $\geq 90$ :",
87 np.sum(a[:, 1:] >= 90, axis=0))
88
89 # 19. Count of subjects  $\geq 90$  for
90 each student
91 print("Roll no:", a[:, 0])
92 print("Count of subjects in which
93 student got marks  $\geq 90$ :",
94 np.sum(a[:, 1:] >= 90, axis=1))
95
96 # 20. Subject 1 marks in ascending
97 order
98 print(np.sort(a[:, 1]))
99
100 # 21. Students who scored between 50
101 and 90 in Subject 1
102 print(a[(a[:, 1] >= 50) & (a[:, 1]
103 <= 90)])
104
105 # 22. Students in range including
106 all (50-90 inclusive shows all
107 except <50)
108 print(a)
109
110 # 23. Index positions of students
111 who scored 79 in Subject 1
112 print(np.where(a[:, 1] == 79))

```